



Chapter 11

Project 3.7: Interim Data Persistence

Our goal is to create files of clean, converted data we can then use for further analysis. To an extent, the goal of creating a file of clean data has been a part of all of the previous chapters. We've avoided looking deeply at the interim results of acquisition and cleaning. This chapter formalizes some of the processing that was quietly assumed in those earlier chapters. In this chapter, we'll look more closely at two topics:

- File formats and data persistence
- The architecture of applications

11.1 Description

In the previous chapters, particularly those starting with [*Chapter 9, Project 3.1: Data Cleaning Base Application*](#), the question of "persistence" was dealt with casually. The previous chapters all wrote the cleaned samples into a file in ND JSON format. This saved delving into the alternatives and the various choices available. It's time to review the previous projects and consider the choice of file format for persistence.

What's important is the overall flow of data from acquisition to analysis. The conceptual flow of data is shown in [*Figure 11.1*](#).

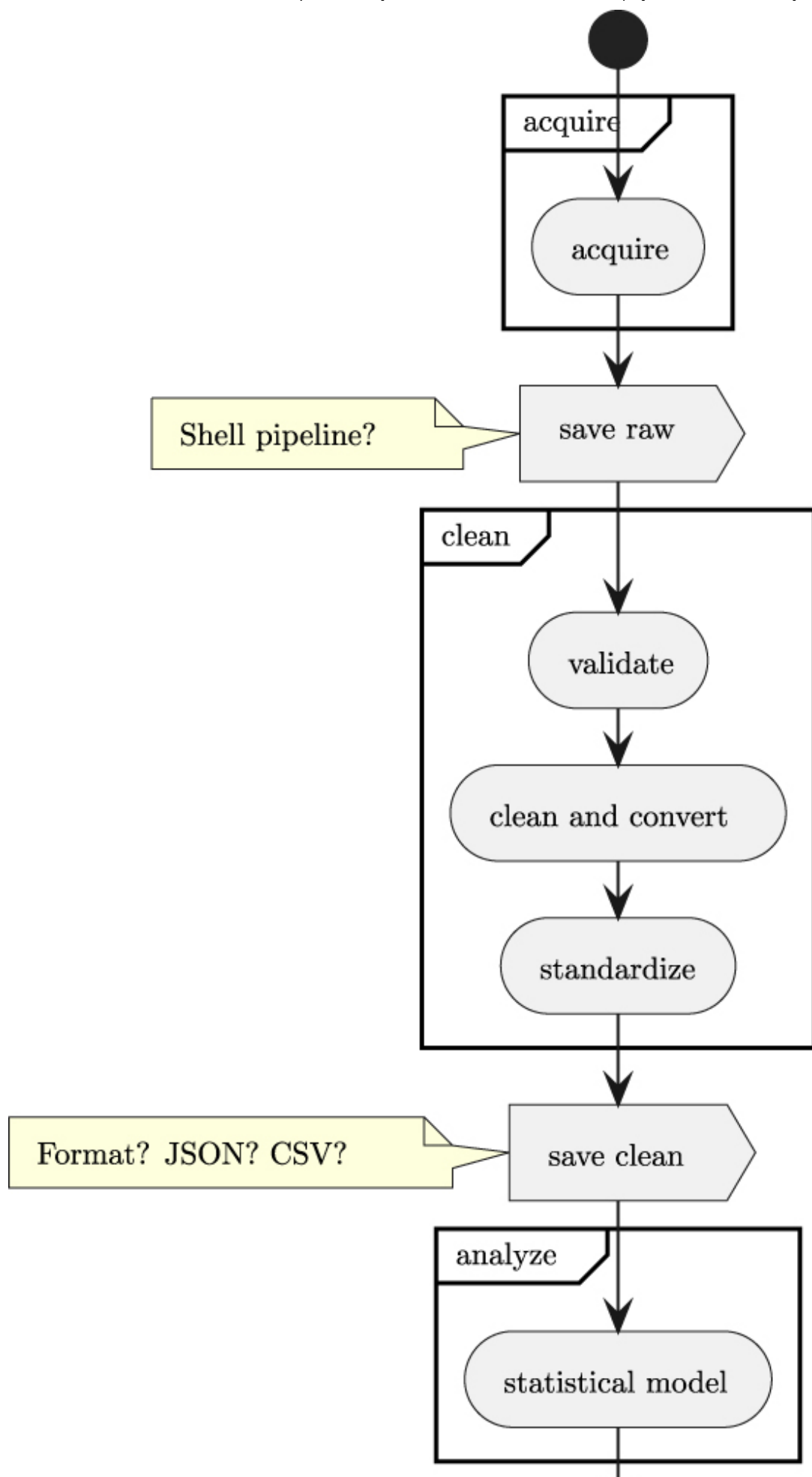




Figure 11.1: Data Analysis Pipeline

This differs from the diagram shown in [*Chapter 2, Overview of the Projects*](#), where the stages were not quite as well defined. Some experience with acquiring and cleaning data helps to clarify the considerations around saving and working with data.

The diagram shows a few of the many choices for persisting interim data. A more complete list of format choices includes the following:

- CSV
- TOML
- JSON
- Pickle
- A SQL database
- YAML

There are others, but this list contains formats that enjoy a direct implementation in Python. Note that YAML is popular but isn't a built-in feature of the Python standard library. Additional formats include protocol buffers (<https://protobuf.dev>) and Parquet (<https://parquet.apache.org>). These two formats require a bit more work to define the structure before serializing and deserializing Python data; we'll leave them out of this discussion.

The CSV format has two disadvantages. The most notable of these problems is the representation of all data types as simple strings. This means any type of conversion information must be offered in metadata outside the CSV file. The **Pydantic** package provides the needed metadata in the form of a class definition, making this format tolerable. The secondary problem is the lack of a deeper structure to the

data. This forces the files to have a flat sequence of primitive attributes.

The JSON format doesn't—directly—serialize datetime or timedelta objects. To make this work reliably, additional metadata is required to deserialize these types from supported JSON values like text or numbers. This missing feature is provided by the **Pydantic** package and works elegantly. A `datetime.datetime` object will serialize as a string, and the type information in the class definition is used to properly parse the string. Similarly, a `datetime.timedelta` is serialized as a float number but converted — correctly — into a `datetime.timedelta` based on the type information in the class definition.

The TOML format has one advantage over the JSON format. Specifically, the TOML format has a tidy way to serialize datetime objects, a capability the JSON library lacks. The TOML format has the disadvantage of not offering a direct way to put multiple TOML documents into a single file. This limits TOML's ability to handle vast datasets. Using a TOML file with a simple array of values limits the application to the amount of data that can fit into memory.

The pickle format can be used with the **Pydantic** package. This format has the advantage of preserving all of the Python-type information and is also very compact. Unlike JSON, CSV, or TOML, it's not human-friendly and can be difficult to read. The `shelve` module permits the building of a handy database file with multiple pickled objects that can be saved and reused. While it's technically possible to execute arbitrary code when reading a pickle file, the pipeline of acquisition and cleansing applications does not involve any unknown agencies providing data of unknown provenance.

A SQL database is also supported by the **Pydantic** package by using an ORM model. This means defining two models in parallel. One model is for the ORM layer (for example, SQLAlchemy) to create table defini-

tions. The other model, a subclass of `pydantic.BaseModel`, uses native **Pydantic** features. The **Pydantic** class will have a `from_orm()` method to create native objects from the ORM layer, performing validation and cleaning.

The YAML format offers the ability to serialize arbitrary Python objects, a capability that makes it easy to persist native Python objects. It also raises security questions. If care is taken to avoid working with uploaded YAML files from insecure sources, the ability to serialize arbitrary Python code is less of a potential security problem.

Of these file formats, the richest set of capabilities seems to be available via JSON. Since we'll often want to record many individual samples in a single file, **newline-delimited (ND) JSON** seems to be ideal.

In some situations — particularly where spreadsheets will be used for analysis purposes — the CSV format offers some value. The idea of moving from a sophisticated Jupyter Notebook to a spreadsheet is not something we endorse. The lack of automated test capabilities for spreadsheets suggests they are not suitable for automated data processing.

11.2 Overall approach

For reference see [*Chapter 9, Project 3.1: Data Cleaning Base Application*](#), specifically [*Approach*](#). This suggests that the `clean` module should have minimal changes from the earlier version.

A cleaning application will have several separate views of the data. There are at least four viewpoints:

- The source data. This is the original data as managed by the upstream applications. In an enterprise context, this may be a transactional database with business records that are precious and part

of day-to-day operations. The data model reflects considerations of those day-to-day operations.

- Data acquisition interim data, usually in a text-centric format. We've suggested using ND JSON for this because it allows a tidy dictionary-like collection of name-value pairs, and supports quite complex Python data structures. In some cases, we may perform some summarization of this raw data to standardize scores. This data may be used to diagnose and debug problems with upstream sources. It's also possible that this data only exists in a shared buffer as part of a pipeline between an acquire and a cleaning application.
- Cleaned analysis data, using native Python data types including `datetime`, `timedelta`, `int`, `float`, and `boolean`. These are supplemented with **Pydantic** class definitions that act as metadata for proper interpretation of the values. These will be used by people to support decision-making. They may be used to train AI models used to automate some decision-making.
- The decision-maker's understanding of the available information. This viewpoint tends to dominate discussions with users when trying to gather, organize, and present data. In many cases, the user's understanding grows and adapts quickly as data is presented, leading to a shifting landscape of needs. This requires a great deal of flexibility to provide the right data to the right person at the right time.

The **acquire** application overlaps with two of these models: it consumes the source data and produces an interim representation. The **clean** application also overlaps two of these models: it consumes the interim representation and produces the analysis model objects. It's essential to distinguish these models and to use explicit, formal mappings between them.

This need for a clear separation and obvious mappings is the primary reason why we suggest including a “builder” method in a model class. Often we've called it something like `from_row()` or `from_dict()` or

something that suggests the model instance is built from some other source of data via explicit assignment of individual attributes.

Conceptually, each model has a pattern similar to the one shown in the following snippet:

-

```
class Example:
    field_1: SomeElementType
    field_2: AnotherType

    @classmethod
    def from_source(cls, source: SomeRowType) -> "Example":
        return Example(
            field_1=transform_1(source),
            field_2=transform_2(source),
        )
```

The transformation functions, `transform1()` and `transform2()`, are often implicit when using `pydantic.BaseModel`. This is a helpful simplification of this design pattern. The essential idea, however, doesn't change, since we're often rearranging, combining, and splitting source fields to create useful data.

When the final output format is either CSV or JSON, there are two helpful methods of `pydantic.BaseModel`. These methods are `dict()` and `json()`. The `dict()` method creates a native Python dictionary that can be used by a `csv.DictWriter` instance to write CSV output. The `json()` method can be used directly to write data in ND JSON format. It's imperative for ND JSON to make sure the `indent` value used by the `json.dump()` function is `None`. Any other value for the `indent` parameter will create multi-line JSON objects, breaking the ND JSON file format.

The **acquire** application often has to wrestle with the complication of data sources that are unreliable. The application should save some his-

tory from each attempt to acquire data and acquire only the "missing" data, avoiding the overhead of rereading perfectly good data. This can become complicated if there's no easy way to make a request for a subset of data.

When working with APIs, for example, there's a `Last-Modified` header that can help identify new data. The `If-Modified-Since` header on a request can avoid reading data that's unchanged. Similarly, the `Range` header might be supported by an API to permit retrieving parts of a document after a connection is dropped.

When working with SQL databases, some variants of the `SELECT` statement permit `LIMIT` and `OFFSET` clauses to retrieve data on separate pages. Tracking the pages of data can simplify restarting a long-running query.

Similarly, the **clean** application needs to avoid re-processing data in the unlikely event that it doesn't finish and needs to be restarted. For very large datasets, this might mean scanning the previous, incomplete output to determine where to begin cleaning raw data to avoid re-processing rows.

We can think of these operations as being "idempotent" in the cases when they have run completely and correctly. We want to be able to run (and re-run) the **acquire** application without damaging intermediate result files. We also want an additional feature of adding to the file until it's correct and complete. (This isn't precisely the definition of "idempotent"; we should limit the term to illustrate that correct and complete files are not damaged by re-running an application.) Similarly, the **clean** application should be designed so it can be run — and re-run — until all problems are resolved without overwriting or reprocessing useful results.

11.2.1 Designing idempotent operations

Ideally, our applications present a UX that can be summarized as "pick up where they left off." The application will check for output files, and avoid destroying previously acquired or cleaned data.

For many of the carefully curated Kaggle data sets, there will be no change to the source data. A time-consuming download can be avoided by examining metadata via the Kaggle API to decide if a file previously downloaded is complete and still valid.

For enterprise data, in a constant state of flux, the processing must have an explicit "as-of date" or "operational date" provided as a run-time parameter. A common way to make this date (or date-and-time) evident is to make it part of a file's metadata. The most visible location is the file's name. We might have a file named `2023-12-31-manufacturing-orders.ndj`, where the as-of date is clearly part of the file name.

Idempotency requires programs in the data acquisition and cleaning pipeline to check for existing output files and avoid overwriting them unless an explicit command-line option permits overwriting. It also requires an application to read through the output file to find out how many rows it contains. This number of existing rows can be used to tailor the processing to avoid re-processing existing rows.

Consider an application that reads from a database to acquire raw data. The "as-of-date" is 2022-01-18, for example. When the application runs and something goes wrong in the network, the database connection could be lost after processing a subset of rows. We'll imagine the output file has 42 rows written before the network failure caused the application to crash.

When the log is checked and it's clear the application failed, it can be re-run. The program can check the output directory and find the file with 42 rows, meaning the application is being run in recovery mode. There should be two important changes to behavior:

- Add a `LIMIT -1 OFFSET 42` clause to the `SELECT` statement to skip the 42 rows already retrieved. (For many databases, `LIMIT -1 OFFSET 0` will retrieve all rows; this can be used as a default value.)
- Open the output file in "append" mode to add new records to the end of the existing file.

These two changes permit the application to be restarted as many times as required to query all of the required data.

For other data sources, there may not be a simple "limit-offset" parameter in the query. This may lead to an application that reads and ignores some number of records before processing the remaining records. When the output file doesn't exist, the offset before processing has a value of zero.

It's important to handle date-time ranges correctly.

It's imperative to make sure date and date-time ranges are properly **half-open intervals**. The starting date and time are included. The ending date and time are excluded.

Consider a weekly extract of data.

One range is 2023-01-14 to 2023-01-21. The 14th is included. The 21st is not included. The next week, the range is 2023-01-21 to 2023-01-28. The 21st is included in this extract.

Using half-open intervals makes it easier to be sure no date is accidentally omitted or duplicated.

Now that we've considered the approach to writing the interim data, we can look at the deliverables for this project.

11.3 Deliverables

The refactoring of existing applications to formalize the interim file formats leads to changes in existing projects. These changes will ripple through to unit test changes. There should not be any acceptance test changes when refactoring the data model modules.

Adding a "pick up where you left off" feature, on the other hand, will lead to changes in the application behavior. This will be reflected in the acceptance test suite, as well as unit tests.

The deliverables depend on which projects you've completed, and which modules need revision. We'll look at some of the considerations for these deliverables.

11.3.1 Unit test

A function that creates an output file will need to have test cases with two distinct fixtures. One fixture will have a version of the output file, and the other fixture will have no output file. These fixtures can be built on top of the `pytest.tmp_path` fixture. This fixture provides a unique temporary directory that can be populated with files needed to confirm that existing files are appended to instead of overwritten.

Some test cases will need to confirm that existing files were properly extended. Other test cases will confirm that the file is properly created when it didn't exist. An edge case is the presence of a file of length zero — it was created, but no data was written. This can be challenging when there is no previous data to read to discover the previous state.

Another edge case is the presence of a damaged, incomplete row of data at the end of the file. This requires some clever use of the `seek()` and `tell()` methods of an open file to selectively overwrite the incomplete final record of the file. One approach is to use the `tell()` method before reading each sample. If an exception is raised by the

file's parser, seek to the last reported `tell()` position, and start writing there.

11.3.2 Acceptance test

The acceptance test scenarios will require an unreliable source of data. Looking back at [*Chapter 4, Data Acquisition Features: Web APIs and Scraping*](#), specifically [*Acceptance tests*](#), we can see the acceptance test suite involves using the `bottle` project to create a very small web service.

There are two aspects to the scenarios, each with different outcomes. The two aspects are:

1. The service or database provides all results or it fails to provide a complete set of results.
2. The working files are not present — we could call this the “clean start” mode — or partial files exist and the application is working in recovery mode.

Since each aspect has two alternatives, there are four combinations of scenarios for this feature:

1. The existing scenario is where the working directory is empty and the API or database works correctly. All rows are properly saved.
2. A new scenario where the working directory is empty and the service or database returns a partial result. The returned rows are saved, but the results are marked as incomplete, perhaps with an error entry in the log.
3. A new scenario where the given working directory has partial results and the API or database works correctly. The new rows are appended to existing rows, leading to a complete result.
4. A new scenario where the given working directory has partial results and the service or database returns a partial result. The cumu-

lative collection of rows are usable, but the results are still marked as incomplete.

A version of the mock RESTful process can return some rows and even after that return 502 status codes. The database version of the incomplete results scenarios is challenging because SQLite is quite difficult to crash at run-time. Rather than try to create a version of SQLite that times out or crashes, it's better to rely on unit testing with a mock database to be sure crashes are handled properly. The four acceptance test scenarios will demonstrate that working files are extended without being overwritten.

11.3.3 Cleaned up re-runnable application design

The final application with the "pick-up-where-you-left-off" feature can be very handy for creating robust, reliable analytic tools. The question of "what do we do to recover?" should involve little (or no) thought.

Creating "idempotent" applications, in general, permits rugged and reliable processing. When an application doesn't work, the root cause must be found and fixed, and the application can be run again to finish the otherwise unfinished work from the failed attempt. This lets analysts focus on what went wrong — and fixing that — instead of having to figure out how to finish the processing.

11.4 Summary

In this chapter, we looked at two important parts of the data acquisition pipeline:

- File formats and data persistence
- The architecture of applications

There are many file formats available for Python data. It seems like newline delimited (ND) JSON is, perhaps, the best way to handle large

files of complex records. It fits well with Pydantic’s capabilities, and the data can be processed readily by Jupyter Notebook applications.

The capability to retry a failed operation without losing existing data can be helpful when working with large data extractions and slow processing. It can be very helpful to be able to re-run the data acquisition without having to wait while previously processed data is processed again.

11.5 Extras

Here are some ideas for you to add to these projects.

11.5.1 Using a SQL database

Using a SQL database for cleaned analytical data can be part of a comprehensive database-centric data warehouse. The implementation, when based on **Pydantic**, requires the native Python classes as well as the ORM classes that map to the database.

It also requires some care in handling repeated queries for enterprise data. In the ordinary file system, file names can have processing dates. In the database, this is more commonly assigned to an attribute of the data. This means multiple time periods of data occupy a single table, distinguished by the “as-of” date for the rows.

A common database optimization is to provide a “time dimension” table. For each date, the associated date of the week, fiscal weeks, month, quarter, and year is provided as an attribute. Using this table saves computing any attributes of a date. It also allows the enterprise fiscal calendar to be used to make sure that 13-week quarters are used properly, instead of the fairly arbitrary calendar month boundaries.

This kind of additional processing isn’t required but must be considered when thinking about using a relational database for analysis

data.

This extra project can use SQLAlchemy to define an ORM layer for a SQLite database. The ORM layer can be used to create tables and write rows of analysis data to those tables. This permits using SQL queries to examine the analysis data, and possibly use complex `SELECT-GROUP BY` queries to perform some analytic processing.

11.5.2 Persistence with NoSQL databases

There are many NoSQL databases available. A number of products like MongoDB use a JSON-based document store. Database engines like PostgreSQL and SQLite3 have the capability of storing JSON text in a column of a database table. We'll narrow our focus onto JSON-based databases as a way to avoid looking at the vast number of databases available.

We can use SQLite3 BLOB columns to store JSON text, creating a NoSQL database using the SQLite3 storage engine.

A small table with two columns: `doc_id`, and `doc_text`, can create a NoSQL-like database. The SQL definition would look like this:

```
CREATE TABLE IF NOT EXISTS document(  
    doc_id INTEGER PRIMARY KEY,  
    doc_text BLOB  
)
```

This table will have a primary key column that's populated automatically with integer values. It has a text field that can hold the serialized text of a JSON document.

The SQLite3 function `json()` should be used when inserting JSON documents:

```
INSERT INTO document(doc_text) VALUES(json(:json_text))
```

This will confirm the supplied value of `json_text` is valid JSON, and will also minimize the storage, removing needless whitespace. This statement is generally executed with the parameter `{"json_text": json.dumps(document)}` to convert a native Python document into JSON text so it can then be persisted into the database.

The attributes of a JSON object can be interrogated using the SQLite `->>` operator to extract a field from a JSON document. A query for a document with a named field that has a specific value will look like this:

```
SELECT doc_text FROM document WHERE doc_text ->> 'field' = :value
```

In the above SQL, the field's name, `field`, is fixed as part of the SQL. This can be done when the schema is designed to support only a few queries. In the more general case, the field name might be provided as a parameter value, leading to a query like the following:

```
SELECT doc_text FROM document WHERE doc_text ->> :name = :value
```

This query requires a small dictionary with the keys "name" and "value", which will provide the field name and field value used to locate matching documents.

This kind of database design lets us write processing that's similar to some of the capabilities of a document store without the overhead of installing a document store database. The JSON documents can be inserted into this document store. The query syntax uses a few SQL keywords as overhead, but the bulk of the processing can be JSON-based interrogation of documents to locate the desired subset of available documents.

The idea here is to use a JSON-based document store instead of a file in ND JSON format. The Document Store interface to SQLite3 should be a

module that can be reused in a JupyterLab Notebook to acquire and analyze data. While unit tests are required for the database interface, there are a few changes to the acceptance test suite required to confirm this changed design.